College of Computer and Information Sciences (CCIS)

Term 252

CS330: Introduction to Operating Systems

Parallel Performance Evaluation Using Process-Based Execution in Linux

Instructor: Dr. Souad Larabi

Team Members:

SHOUG FAWAZ ABDULLAH ALOMRAN - 223410392

LAYAN HESHAM RASHED BINDAYEL - 223410104

ARYAM YAHYA ALMANSOUR - 223410070

DANAH ABDULAZIZ ABDULMOHSEN ALTUWAIJRI - 223410151

RANA ABDULMALIK HAMAD ALFARIS - 223410567

TABLE OF CONTENTS

1. Objective **2**
2. Introduction **3**
3. Methodology **3**
4. Experimental Setup **4**
5. Results **4**
5.1 Summary of Results 4
5.2 Performance Graphs 5
5.2.1 Execution Time vs. Number of Threads 5
5.2.2 Speedup vs. Number of Threads 5
5.2.3 Percentage Improvement vs. Number of Threads 5
5.3 Explanation of Metrics 5
5.4 Interpretation of Results 6
5.4.1 Results Table 6
5.4.2 Sample Execution Output 8
6. Performance Analysis **8**
7. Maximum Effective Thread Count **9**
8. Conclusion **9**

1. Objective

The objective of this project is to evaluate the performance impact of multithreading on a multi-core system. Specifically, the program is modified to compute the sum of cubes of array elements and measure execution time using different numbers of threads (1, 2, 4, 6, and 8). The goal is to analyze speedup, efficiency, and scalability of parallel execution.

2. Introduction

Modern computing systems rely on multi-core processors to achieve higher performance through parallel execution. Multithreading enables programs to divide computational tasks into smaller units that can be processed concurrently, thereby reducing execution time.

In this project, a Java program was analyzed and modified to evaluate the performance impact of multithreading. The original implementation computed the sum of array elements using different thread counts. As required, the program was modified to compute the sum of cubes of array elements, and performance was measured using 1, 2, 4, 6, and 8 threads .

The objective of this phase is to measure execution time, calculate speedup and percentage improvement, and analyze how performance scales with increasing thread count.

3. Methodology

The methodology involves modifying the original single-threaded program to support multithreading and measuring its performance. The array is divided into equal segments, and each segment is assigned to a separate thread. Each thread computes the sum of cubes for its assigned portion, and the results are combined.

Performance is evaluated by measuring execution time and calculating speedup and percentage improvement relative to the single-threaded execution.

The program divides the input array into equal segments based on the number of threads. Each thread is responsible for computing the sum of cubes for its assigned segment. Once all threads complete execution, their partial results are combined to produce the final result.

The following modifications were implemented:

- The computation was changed from summation to sum of cubes
- Array values were restricted to integers less than 100
- Support was added for multiple thread configurations (1, 2, 4, 6, and 8 threads)
- Execution time was measured for each configuration

This approach allows multiple threads to execute concurrently, improving performance when system resources are effectively utilized.

4. Experimental Setup

The experiments were conducted on the following system:

- Processor: Apple M2
- Number of cores: 8
- RAM: 8 GB
- Operating System: macOS Tahoe 26.3.1
- Java Version: OpenJDK 25 LTS (Temurin)

Each configuration was executed multiple times, and the results were averaged to ensure accuracy and consistency.

5. Results

5.1 Summary of Results

Threads	Execution Time (ms)	Speedup	% Improvement
1	3.921	1.00	0.00
2	4.560	0.86	-16.30
4	3.022	1.30	22.94

6	0.747	5.25	80.96
8	0.537	7.30	86.31

5.2 Performance Graphs

5.2.1 Execution Time vs. Number of Threads
Insert graph
Explanation: Overall, the majority of multithreaded processes i.e. threads 4,6,8 often result in reduced execution time in ms, relative to the baseline of a heavyweight single-threaded process. For instance, and based on the summarized results pictured above, a minor decrease in execution time is illustrated between a single-threaded process and a process composed of 4 threads. Additionally, a correlation is consistently being established between reduced execution time *(ms)* and the increase in the number of the threads within a process, where a significant drop of approximately 3ms was observed as the thread number increased to 6 and 8. However, a major tradeoff can be highlighted regarding dual-threaded process models. Its execution time (ms) revealed a noticeable increase, whereby thread 1 yielded a time of 3.921ms while thread 2 yielded a time of 4.560ms. This had occurred as a result of the pure overhead caused by the creation, scheduling and synchronization of the thread that outweighed its light workload

5.2.2 Speedup vs. Number of Threads
Insert graph
Explanation:

5.2.3 Percentage Improvement vs. Number of Threads
Insert Graph
Explanation:

5.3 Explanation of Metrics

The results are evaluated using three main metrics: execution time, speedup, and percentage improvement.

- **Execution Time (ms):** This represents the time required to complete the computation. Lower values indicate better performance.
- **Speedup:** Speedup measures how much faster the multithreaded version is compared to the single-threaded baseline. It is calculated as:
  - Speedup = (Execution time with 1 thread) / (Execution time with N threads)
  - For example, at 8 threads: Speedup = 3.921 / 0.537 ≈ 7.30
  - This means the program runs approximately 7.3 times faster than the single-threaded version.
- **Percentage Improvement**: This metric shows how much performance has improved relative to the baseline: $\% \, Improvement = ((T_1 - T_N) / T_1) \times 100$

  where:
  - T_1 = execution time with 1 thread
  - T_N = execution time with N threads

5.4 Interpretation of Results

The negative percentage improvement observed at 2 threads (-16.30%) indicates that performance has degraded compared to the single-threaded execution.

This occurs because:

- The overhead of creating and managing threads exceeds the benefit of parallel execution
- Additional costs such as context switching and synchronization reduce efficiency
- The workload is not large enough to benefit from parallelism at this level

As a result, the execution time increases from 3.921 ms (1 thread) to 4.560 ms (2 threads), leading to a negative improvement value.

In contrast, positive percentage improvements at higher thread counts (4, 6, and 8 threads) indicate that the benefits of parallel execution outweigh the overhead. For example:

- At 4 threads, a 22.94% improvement shows moderate performance gain
- At 6 threads, an 80.96% improvement indicates significant acceleration
- At 8 threads, the improvement reaches 86.31%, demonstrating near-optimal utilization of system resources

These results confirm that multithreading becomes effective only when the workload is sufficiently distributed and the number of threads aligns with the available CPU cores.

5.4.1 Results Table

Run	Threads	Execution Time (ms)	Speedup	% Improvement
1	1	3.921	1	0
1	2	4.56	0.86	-16.3
1	4	3.022	1.3	22.94
1	6	0.747	5.25	80.96
1	8	0.537	7.3	86.31
2	1	3.671	1	0
2	2	4.207	0.87	-14.61
2	4	3.077	1.19	16.16
2	6	0.634	5.79	82.72
2	8	0.788	4.66	78.54
3	1	3.813	1	0
3	2	4.273	0.89	-12.06
3	4	3.171	1.2	16.85
3	6	0.685	5.56	82.03
3	8	0.588	6.48	84.57
4	1	3.69	1	0
4	2	4.252	0.87	-15.23
4	4	3.116	1.18	15.56
4	6	0.678	5.44	81.63
4	8	0.773	4.77	79.05
5	1	3.759	1	0
5	2	4.244	0.89	-12.89
5	4	3.139	1.2	16.49
5	6	0.592	6.35	84.26
5	8	0.611	6.15	83.74
6	1	3.702	1	0
6	2	4.217	0.88	-13.89
6	4	3.113	1.19	15.93
6	6	0.708	5.23	80.87
6	8	1.013	3.66	72.65
7	1	3.704	1	0
7	2	4.362	0.85	-17.76
7	4	3.071	1.21	17.09
7	6	0.848	4.37	77.11
7	8	0.567	6.53	84.68
8	1	3.74	1	0
8	2	4.771	0.78	-27.57
8	4	2.823	1.32	24.51
8	6	0.616	6.07	83.53
8	8	0.571	6.55	84.74
9	1	3.724	1	0
9	2	4.24	0.88	-13.84
9	4	3.446	1.08	7.49
9	6	0.631	5.9	83.05
9	8	0.65	5.73	82.56
10	1	3.924	1	0
10	2	4.205	0.93	-7.16
10	4	3.139	1.25	20.01
10	6	0.758	5.18	80.68
10	8	0.639	6.14	83.71
11	1	3.725	1	0
11	2	4.347	0.86	-16.7
11	4	3.119	1.19	16.27
11	6	0.772	4.82	79.27
11	8	0.515	7.24	86.18
12	1	3.68	1	0
12	2	4.348	0.85	-18.13
12	4	3.094	1.19	15.94
12	6	0.707	5.2	80.78
12	8	0.533	6.9	85.51

5.4.2 Sample Execution Output

```
Threads  Execution Time (ms)  Speedup    % Improvement
1        3.759                1.00       0.00
2        4.244                0.89       -12.89
4        3.139                1.20       16.49
6        0.592                6.35       84.26
8        0.611                6.15       83.74

Machine Specifications:
Processor model: Apple M2
Number of cores: 8
RAM: 8 GB
Operating system: macOS Tahoe 26.3.1
Java version: OpenJDK 25 LTS (Temurin)
```

```
Threads  Execution Time (ms)  Speedup    % Improvement
1        3.924                1.00       0.00
2        4.205                0.93       -7.16
4        3.139                1.25       20.01
6        0.758                5.18       80.68
8        0.639                6.14       83.71

Machine Specifications:
Processor model: Apple M2
Number of cores: 8
RAM: 8 GB
Operating system: macOS Tahoe 26.3.1
Java version: OpenJDK 25 LTS (Temurin)
```

```
Threads  Execution Time (ms)  Speedup    % Improvement
1        3.921                1.00       0.00
2        4.560                0.86       -16.30
4        3.022                1.30       22.94
6        0.747                5.25       80.96
8        0.537                7.30       86.31

Machine Specifications:
Processor model: Apple M2
Number of cores: 8
RAM: 8 GB
Operating system: macOS Tahoe 26.3.1
Java version: OpenJDK 25 LTS (Temurin)
```

6. Performance Analysis

The results demonstrate that multithreading can significantly improve performance; however, the improvement is dependent on the number of threads and system capabilities.

When using a **single** thread, the program executes sequentially, resulting in the highest execution time. This serves as the baseline for comparison.

With **two** threads, performance decreases slightly. This is due to thread management overhead, including thread creation and context switching, which outweighs the benefits of parallel execution for this workload.

At **four** threads, performance begins to improve. The workload is distributed more effectively, allowing better utilization of available CPU resources.

A significant improvement is observed at **six** threads. At this point, parallel execution becomes efficient, and the benefits of dividing the workload outweigh the overhead costs.

The best performance is achieved with **eight** threads. Since the system has eight cores, each thread can be executed on a separate core, maximizing CPU utilization and minimizing idle time.

These results illustrate that performance improves as the number of threads increases up to the number of available cores. Beyond this point, performance may degrade due to increased overhead and resource contention.

7. Maximum Effective Thread Count

The optimal performance was achieved using eight threads, which corresponds to the number of available CPU cores on the system.

This demonstrates that the most effective thread count is typically equal to the number of cores. Increasing the number of threads beyond this limit is expected to reduce performance due to additional scheduling overhead and context switching.

8. Conclusion

This project highlights the importance of multithreading in improving application performance on multi-core systems. By distributing computational tasks across multiple threads, execution time can be significantly reduced.

However, the results also demonstrate that multithreading must be applied carefully. Excessive threads can introduce overhead that negatively impacts performance.

The experiment confirms that optimal performance is achieved when the number of threads matches the number of available CPU cores. This balance ensures efficient utilization of system resources while minimizing overhead.

Overall, this project provides practical insight into parallel programming, performance measurement, and scalability in modern computing systems.